CRYPTOGRAPHY COURSE

Analytical Problem Solving

CO2-AT-1

NAME:S.DHIVYASRI

REG NO:192511246

1. RSA Encryption Analysis

Given p=11, q=13

a. Compute n φ(n)\

b. Choose e=7 and find the private key d

c. Encrypt the message M=9

d. Decrypt the ciphertext and verify the result

e. Analyze how changing e affects security

ANSWER:

RSA Encryption Analysis

Given:

- p = 11
- q = 13
- e = 7
- Message (M) = 9

(a) Compute n and φ(n)

The value of n is calculated using:

$n = p \times q$

$= 11 \times 13$

$= 143$

The value of Euler's Totient Function is:

$\varphi(n) = (p - 1)(q - 1)$

$= (11 – 1)(13 – 1)$

$= 10 \times 12$

$= 120$

Answer:

- $n = 143$
- $\varphi(n) = 120$

(b) Choose e = 7 and find the private key d

The private key d is calculated using the formula:

$d \times e \equiv 1 \pmod{\varphi(n)}$

Substituting the values:

$7 \times d \equiv 1 \pmod{120}$

Find the multiplicative inverse of 7 modulo 120.

$7 \times 103 = 721$

$721 \div 120 = 6$ remainder 1

Therefore,

$721 \equiv 1 \pmod{120}$

Hence,

$d = 103$

Public Key = (7, 143)

Private Key = (103, 143)

(c) Encrypt the message M = 9

RSA Encryption Formula:

$C = M^e \bmod n$

Substitute the values:

$C = 9^7 \bmod 143$

Calculation:

$9^2 = 81$

$9^4 = 81^2 = 6561$

$6561 \bmod 143 = 126$

$9^7 = 9^4 \times 9^2 \times 9$

$= 126 \times 81 \times 9$

$= 10206 \times 9$

$10206 \bmod 143 = 53$

$53 \times 9 = 477$

$477 \bmod 143 = 48$

Therefore,

Ciphertext (C) = 48

(d) Decrypt the ciphertext

RSA Decryption Formula:

$M = C^d \bmod n$

Substitute the values:

$M = 48^{103} \bmod 143$

Using modular exponentiation,

$48^{103} \bmod 143 = 9$

Therefore,

Decrypted Message = 9

Verification:

Original Message = 9

Decrypted Message = 9

Since both values are the same, the encryption and decryption process is verified successfully.

(e) Analyze how changing e affects security

- The public exponent e must be relatively prime to φ(n).
- A smaller value of e makes encryption faster but may reduce security if proper padding is not used.
- A larger value of e increases encryption time but provides better resistance against some attacks.
- The value 65537 is commonly used because it offers a good balance between security and efficiency.

- The overall security of RSA mainly depends on the difficulty of factoring the large value of n, while the choice of e mainly affects encryption efficiency and protection against specific attacks.

Final Answer

- n = 143
- φ(n) = 120
- Public Key = (7, 143)
- Private Key = (103, 143)
- Ciphertext = 48
- Decrypted Message = 9
- Verification: Original Message = Decrypted Message = 9

2. Block Cipher Mode Evaluation

Given plaintext blocks:

P=[1010,1010,1111,1010], Key = K=1100

a. Encrypt using ECB and CBC modes (Assume IV = 0000)

b. Compare the ciphertext outputs

c. Analyze which mode hides patterns better and explain why

ANSWER:

Block Cipher Mode Evaluation

Given:

- Plaintext Blocks (P) = [1010, 1010, 1111, 1010]
- Key (K) = 1100
- Initialization Vector (IV) = 0000

Assumption: Encryption is performed using the XOR operation.

Encryption Formula:

Ciphertext = Plaintext ⊕ Key

(a) Encrypt using ECB and CBC Modes

ECB (Electronic Code Book) Mode

In ECB mode, each plaintext block is encrypted independently using the same key.

Block 1

$P_1 = 1010$

$K = 1100$

$C_1 = 1010 \oplus 1100 = \mathbf{0110}$

Block 2

$P_2 = 1010$

$K = 1100$

$C_2 = 1010 \oplus 1100 = \mathbf{0110}$

Block 3

$P_3 = 1111$

$K = 1100$

$C_3 = 1111 \oplus 1100 = \mathbf{0011}$

Block 4

$P_4 = 1010$

$K = 1100$

$C_4 = 1010 \oplus 1100 = \mathbf{0110}$

ECB Ciphertext

ECB = [0110, 0110, 0011, 0110]

CBC (Cipher Block Chaining) Mode

In CBC mode, each plaintext block is first XORed with the previous ciphertext block before encryption.

Block 1

IV = 0000

$P_1 \oplus IV$

1010 ⊕ 0000 = 1010

Encrypt:

1010 ⊕ 1100 = **0110**

C_1 = **0110**

Block 2

Previous Ciphertext = 0110

$P_2 \oplus C_1$

1010 ⊕ 0110 = 1100

Encrypt:

1100 ⊕ 1100 = **0000**

C_2 = **0000**

Block 3

Previous Ciphertext = 0000

$P_3 \oplus C_2$

1111 ⊕ 0000 = 1111

Encrypt:

1111 ⊕ 1100 = **0011**

C_3 = **0011**

Block 4

Previous Ciphertext = 0011

$P_4 \oplus C_3$

1010 ⊕ 0011 = 1001

Encrypt:

1001 ⊕ 1100 = **0101**

C_4 = **0101**

CBC Ciphertext

CBC = [0110, 0000, 0011, 0101]

(b) Compare the Ciphertext Outputs

Plaintext Block	ECB Ciphertext	CBC Ciphertext
1010	0110	0110
1010	0110	0000
1111	0011	0011
1010	0110	0101

Comparison:

- In ECB mode, identical plaintext blocks produce identical ciphertext blocks.
- In CBC mode, identical plaintext blocks produce different ciphertext blocks because each block depends on the previous ciphertext.

(c) Analyze Which Mode Hides Patterns Better

ECB Mode:

- Encrypts each block independently.
- Repeated plaintext blocks produce repeated ciphertext blocks.
- Patterns in the plaintext remain visible.
- Therefore, ECB is less secure for encrypting large amounts of data.

CBC Mode:

- Each plaintext block is combined with the previous ciphertext block before encryption.
- Even if two plaintext blocks are identical, their ciphertext blocks are usually different.
- This hides patterns and provides better security than ECB.

Conclusion:

CBC mode hides patterns better than ECB mode because every ciphertext block depends on both the plaintext block and the previous ciphertext block. Hence, CBC provides stronger confidentiality and is more secure for practical applications.

Final Answer

- **ECB Ciphertext:** [0110, 0110, 0011, 0110]
- **CBC Ciphertext:** [0110, 0000, 0011, 0101]
- **Better Mode:** CBC
- **Reason:** CBC hides repeated plaintext patterns by using the previous ciphertext block during encryption, making it more secure than ECB.

3. DES vs 3-DES vs Blowfish Performance Study

Given:

a. DES key size = 56 bits, time = 2 ms/block

b. 3-DES key size = 168 bits, time = 6 ms/block

c. Blowfish key size = 128 bits, time = 1 ms/block

ANSWER:

DES vs 3-DES vs Blowfish Performance Study

Given:

- **DES:** Key Size = 56 bits, Time = 2 ms/block
- **3-DES:** Key Size = 168 bits, Time = 6 ms/block
- **Blowfish:** Key Size = 128 bits, Time = 1 ms/block
- **Number of Data Blocks = 1000**

(i) Calculate Total Encryption Time for Each Algorithm

Formula:

Total Encryption Time = Time per Block × Number of Blocks

1. DES

Time per block = 2 ms

Number of blocks = 1000

Total Time = 2 × 1000

= **2000 ms**

= **2 seconds**

---

2. 3-DES

Time per block = 6 ms

Number of blocks = 1000

Total Time = 6 × 1000

= **6000 ms**

= **6 seconds**

---

3. Blowfish

Time per block = 1 ms

Number of blocks = 1000

Total Time = 1 × 1000

= **1000 ms**

= **1 second**

Total Encryption Time Comparison

Algorithm	Time per Block	Total Time for 1000 Blocks
DES	2 ms	2000 ms (2 s)
3-DES	6 ms	6000 ms (6 s)
Blowfish	1 ms	1000 ms (1 s)

(ii) Analyze Trade-offs Between Security and Performance

DES

- Uses a **56-bit key**.
- Provides the **lowest security** because the key size is small.
- Faster than 3-DES but slower than Blowfish.
- Suitable only for basic or legacy applications.

3-DES

- Uses a **168-bit key**.
- Provides the **highest security** among the three algorithms.
- Slowest encryption because DES is performed three times.
- Suitable for applications where security is more important than speed.

Blowfish

- Uses a **128-bit key**.
- Provides **strong security** with the fastest encryption speed.
- Requires the least encryption time.
- Suitable for applications requiring both high security and fast performance.

(iii) Recommend the Best Algorithm for Secure Real-Time Communication

For secure real-time communication, **Blowfish** is the best choice because:

- It encrypts data the fastest (**1 second for 1000 blocks**).
- It provides strong security with a **128-bit key**.
- It has lower processing time, making it suitable for real-time applications such as video streaming, voice communication, and secure messaging.

Although **3-DES** provides higher security, its encryption time (**6 seconds**) is much longer, making it less suitable for real-time communication.

Conclusion

- **DES:** Moderate speed but low security.
- **3-DES:** Highest security but slowest performance.
- **Blowfish:** Best balance of speed and security, making it the most suitable algorithm for secure real-time communication.

Final Answer

Algorithm	Key Size	Time for 1000 Blocks	Security	Performance
DES	56 bits	2000 ms (2 s)	Low	Moderate
3-DES	168 bits	6000 ms (6 s)	Very High	Slow

Algorithm	Key Size	Time for 1000 Blocks	Security	Performance
Blowfish	128 bits	1000 ms (1 s)	High	Fast

Recommended Algorithm: Blowfish

Reason: It offers strong security with the fastest encryption time, making it the best choice for secure real-time communication.

4. Diffie-Hellman Key Exchange Analysis

Given:

a. p=23, g=5

b. User A private key = 6

c. User B private key = 15

d. Compute public keys for A and B

e. Compute the shared secret key

f. Demonstrate how a Man-in-the-Middle attacker could alter the

exchange

g. Suggest a secure method to prevent the attack

ANSWER:

Diffie-Hellman Key Exchange Analysis

Given:

- Prime number, **p = 23**
- Generator, **g = 5**
- User A Private Key = **6**
- User B Private Key = **15**

(d) Compute Public Keys for A and B

Formula:

Public Key = g^(Private Key) mod p

User A Public Key

$$A = 5^6 \bmod 23$$

$$5^6 = 15625$$

$$15625 \bmod 23 = 8$$

Public Key of User A = 8

User B Public Key

$$B = 5^{15} \bmod 23$$

Using modular exponentiation,

$$5^{15} \bmod 23 = 19$$

Public Key of User B = 19

(e) Compute the Shared Secret Key

User A Computes

$$K = B^a \bmod p$$

$$K = 19^6 \bmod 23$$

$$K = 2$$

User B Computes

$$K = A^b \bmod p$$

$$K = 8^{15} \bmod 23$$

$$K = 2$$

Shared Secret Key = 2

Both users obtain the same secret key.

(f) Demonstrate a Man-in-the-Middle (MITM) Attack

1. User A sends public key **8** to User B.
2. The attacker intercepts the message and replaces **8** with the attacker's own public key.
3. User B receives the attacker's public key instead of A's public key.
4. Similarly, when User B sends public key **19**, the attacker intercepts it and sends another fake public key to User A.
5. User A and User B each establish separate secret keys with the attacker instead of with each other.
6. The attacker can decrypt, read, modify, and re-encrypt all messages without either user knowing.

(g) Suggest a Secure Method to Prevent the Attack

The following methods can prevent a Man-in-the-Middle attack:

- Use **Digital Signatures** to verify the identity of both users.
- Use **Digital Certificates (PKI)** issued by a trusted Certificate Authority.
- Use authenticated versions of the Diffie-Hellman protocol, such as **Authenticated Diffie-Hellman** or **TLS (Transport Layer Security)**.
- Verify public keys before establishing the shared secret.
- Use mutual authentication to ensure both users are communicating with the intended party.

Conclusion

- Public Key of User A = **8**
- Public Key of User B = **19**
- Shared Secret Key = **2**
- A Man-in-the-Middle attacker can replace public keys and establish separate secret keys with both users.
- Using digital signatures, certificates, and authenticated key exchange protocols prevents this attack.

Final Answer

Parameter	**Value**
Prime Number (p)	23
Generator (g)	5
User A Private Key	6
User B Private Key	15
User A Public Key	8
User B Public Key	19
Shared Secret Key	2
Attack	Man-in-the-Middle (MITM)
Prevention	Digital Signatures, Digital Certificates, TLS/Authenticated Diffie-Hellman

5. ECC vs RSA Comparison

Given:

a. RSA key size = 1024 bits

b. ECC key size = 160 bits (equivalent security)

c. Device processing capacity = limited (IoT device)

d. Analyze computational cost for both systems

e. Estimate which consumes less power and memory

f. Justify which cryptosystem is better for the given scenario with

reasoning

ANSWER:

ECC vs RSA Comparison

Given:

- RSA Key Size = **1024 bits**
- ECC Key Size = **160 bits** (Equivalent Security)
- Device = **IoT Device (Limited Processing Capacity)**

(d) Analyze Computational Cost for Both Systems

RSA

- Uses a **1024-bit key** to provide the required level of security.
- Requires large integer arithmetic during encryption and decryption.
- Performs more mathematical computations.
- Therefore, RSA has a **higher computational cost**.

ECC (Elliptic Curve Cryptography)

- Uses a much smaller **160-bit key** while providing security equivalent to RSA 1024-bit.
- Requires fewer mathematical operations.
- Performs encryption and decryption faster than RSA on limited-resource devices.
- Therefore, ECC has a **lower computational cost**.

(e) Estimate Which Consumes Less Power and Memory

RSA

- Larger key size (1024 bits).
- Requires more memory to store keys.
- Consumes more CPU resources during encryption and decryption.
- Uses more battery power.

ECC

- Smaller key size (160 bits).
- Requires less memory for storing keys.
- Performs computations more efficiently.
- Consumes less processor power and battery energy.

Comparison Table

Feature	RSA	ECC
Key Size	1024 bits	160 bits
Computational Cost	High	Low
Memory Usage	High	Low
Power Consumption	High	Low
Processing Speed	Slower	Faster

(f) Justify Which Cryptosystem is Better for the Given Scenario

For an IoT device with limited processing capacity, **ECC (Elliptic Curve Cryptography)** is the better choice because:

- It provides the same security as RSA using a much smaller key.
- It requires less memory to store keys.
- It consumes less processor power.
- It reduces battery usage.
- It performs cryptographic operations faster.
- It is well suited for embedded systems, wireless sensors, smart devices, and Internet of Things (IoT) applications.

Although RSA is widely used, its larger key size increases computation time, memory usage, and power consumption, making it less suitable for resource-constrained devices.

Conclusion

- **RSA:** Higher computational cost, larger memory usage, and higher power consumption.
- **ECC:** Lower computational cost, lower memory usage, and lower power consumption.

- **Best Choice: ECC**, because it provides equivalent security with smaller keys and is ideal for IoT devices.

Final Answer

Parameter	**RSA**	**ECC**
Key Size	1024 bits	160 bits
Equivalent Security	Yes	Yes
Computational Cost	High	Low
Memory Consumption	High	Low
Power Consumption	High	Low
Suitable for IoT	No	Yes

Recommended Cryptosystem: ECC (Elliptic Curve Cryptography)

Reason: ECC provides the same level of security as RSA while using a much smaller key size. It requires less computation, memory, and power, making it the best choice for IoT devices with limited processing capacity.